VC

AI-Based Face Recognition for VC Cabin Entry

Faculty and Student Identification System

AI Prototype • MCA Project

Rajat Sharma • MCA-FS • RU-26-01267

Software-only prototype — detects, recognizes, logs and notifies.

No door or lock is controlled; the human makes the access decision.

1. Problem Statement & Objective

A VC (Vice-Chancellor) cabin entry currently has no reliable record of who entered and when. Manual register entries are slow, inaccurate and easy to forge. The objective is to build a system that identifies students and faculty at the entry point automatically.

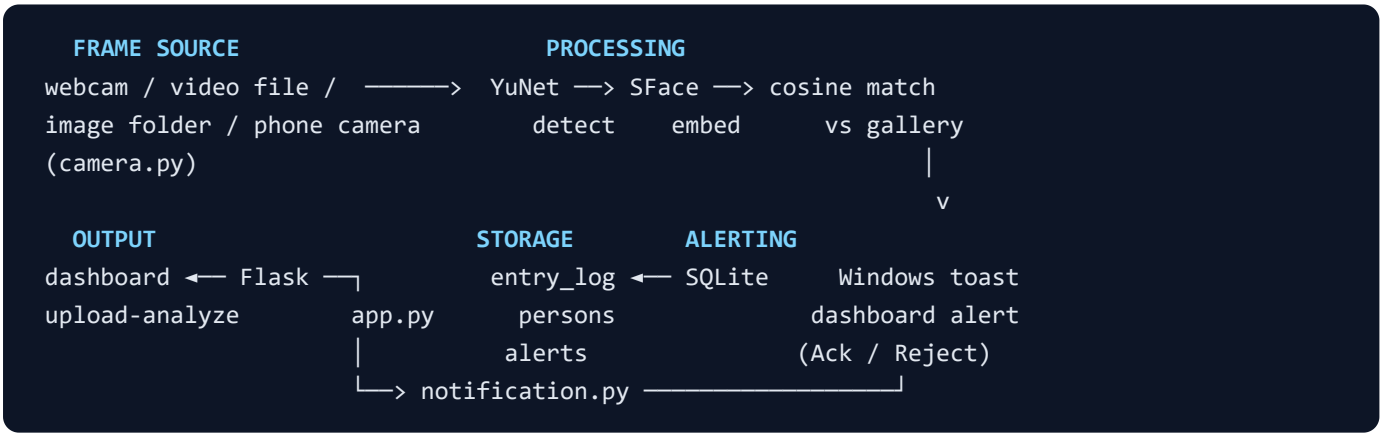
> **Scope boundary (important):** this prototype is **software only**. It detects a face, decides who it is, records it, and raises an alert. It **never opens a door or controls any lock** — the security guard / staff member reviews the alert and physically grants or denies access.

Objectives

- Detect faces reliably at the entry point
- Recognize enrolled students and faculty with a measured confidence
- Log every entry with identity, department, ID, time and result
- Notify staff instantly when a visitor requires attention
- Work with or without a physical webcam (video, images, or phone camera)

2. System Architecture

The system runs entirely on one machine. A background thread pulls frames, a Flask server serves the dashboard, and everything is stored in SQLite.



Module	Responsibility
`camera.py`	Frame sources: webcam index, video file, image folder, or a live network stream (phone camera). Includes a low-latency MJPEG reader.
`face_detection.py`	YuNet face detection wrapper (ONNX).
`face_recognition.py`	SFace 128-D embedding + cosine best-match against the gallery, threshold 0.363.
`pipeline.py`	Background thread: read → detect → match → log → notify, with duplicate suppression.

`database.py`	SQLite: persons (with consent), entry log, alerts with acknowledge/reject decisions.
`enrollment.py`	Consent-gated registration; multiple photos averaged into one embedding.
`notification.py`	Windows toast notification (fire-and-forget, can be disabled).
`app.py`	Flask dashboard, enrollment API, image-upload analysis, live MJPEG camera feed.
`evaluate.py`	Confusion-matrix evaluation → `reports/evaluation.csv`.

3. Technology Stack

Layer	Choice	Why
Face detection	OpenCV **YuNet** (ONNX, 2023-Mar)	Accurate on small/angled faces, fast on CPU
Face recognition	OpenCV **SFace** (ONNX, 2021-Dec)	Compact 128-D embeddings, vendor-published threshold
Similarity	Cosine similarity, threshold **0.363**	OpenCV's published operating point for SFace
Backend	**Flask** (Python)	Simple, local, no external services
Storage	**SQLite**	Zero-setup, file-based, fully local
Frontend	Server-rendered HTML + vanilla JS	No build step, no framework, live polling
Models	OpenCV Model Zoo (ONNX)	Openly licensed, runs offline via OpenCV DNN
Language	Python 3	De-facto standard for OpenCV

> **No cloud, no internet required at runtime.** Inference runs locally through OpenCV DNN. All personal data stays on the machine.

4. How the Pipeline Works

1. **Capture a frame.** From the configured source (webcam, video, image folder, or phone camera over Wi-Fi).
2. **Detect faces (YuNet).** Any face found in the frame is a candidate. If no face is found, the frame is skipped silently.
3. **Generate an embedding (SFace).** The face is *aligned* using its 5 landmarks, then encoded into a 128-D vector that is L2-normalized.
4. **Match against the gallery.** The embedding is compared with every enrolled person using cosine similarity. Above **0.363** it is a match; below it, the system refuses to guess.
5. **Log the entry.** Name, category (Student/Faculty), department, ID, timestamp, confidence and result are written to `entry_log`, along with a cropped snapshot of the face.
6. **Notify staff.** A desktop toast plus a dashboard alert is raised. Staff then **Acknowledge** or **Reject** it.

Duplicate suppression

A 15-second cooldown (`VC_COOLDOWN`) prevents the same person (or the same unknown visitor) from being logged repeatedly while they stand in front of the camera.

What happens when a face is not recognized

The entry is logged as `UNKNOWN` with the raw cosine score. The system **never guesses an identity** — a low-confidence frame is always flagged for manual verification rather than assigned to the nearest enrolled person.

5. Dashboard Features

Screen	What it shows
Dashboard	Four stat cards (Total Visitors, Recognized, Students, Staff), a compact **live camera preview** with pipeline counters, an upload panel with the 5-step recognition workflow, the recognition result card, pending notifications, and the recent visitors table.
Visitor Logs	Full entry history — photo, visitor, category, department, ID, time, result and confidence.
Students & Staff	All enrolled identities with status, and a deactivate action.
Notifications	Pending visitor alerts with **Acknowledge** / **Reject** buttons, plus a "recently resolved" audit trail.
Register Person	Consent-gated enrollment: name, category, department, ID and one or more photos. Multiple photos are averaged into a single embedding for accuracy.

Two ways to use it

- **Live monitoring** — point a webcam (or a phone camera acting as one) at the entry; the dashboard shows the feed and logs people as they pass.
- **Upload & analyze** — drop in a still image and get an instant identity verdict. Useful when no camera is available.

Notification handling

Each alert can be **Acknowledged** (handled/allowed) or **Rejected** (denied). Both actions clear the notification and record the decision for audit. If nobody acts on an alert, it **clears itself after 24 hours**, so the notification list never grows without bound.

6. Live Camera — Phone as a Webcam

The built-in webcam on this machine was not available, so the system was built to accept a phone camera as a live source. Running an app such as **IP Webcam** or **DroidCam** on the phone exposes the camera as an MJPEG video stream over Wi-Fi, which the system consumes directly.

```
py app.py --source http://<phone-ip>:8080/video
```

> **Engineering note:** OpenCV's FFMPEG backend buffers HTTP streams heavily — a phone camera serving ~15 fps was read at under **0.25 fps** through `VideoCapture`. A dedicated MJPEG multipart reader (`camera.MjpegStream`) was written to parse the stream directly, raising throughput to ~**4.3 fps** at 640×480. The pipeline also auto-reconnects if the phone drops off the network.

7. Performance Evaluation

Self-test (re-identification)

Metric	Value	Metric	Value
Samples	2	True Positives (TP)	2
False Positives (FP)	0	False Negatives (FN)	0
Accuracy	100%	Precision	100%
Recall	100%	F1 Score	100%

Live video evaluation (20 probe frames from the demo clip)

20 frames were sampled at regular intervals from the actual entry video and run through the full pipeline:

10
CORRECTLY RECOGNIZED

10
CORRECTLY NOT MATCHED

12
FACES DETECTED IN TOTAL

Scenario	Outcome
Enrolled person in frame	Recognized — cosine 0.37 to 0.67, comfortably above the 0.363 threshold
Un-enrolled background face	Rejected — cosine near zero, logged as UNKNOWN for manual review
Frames with no face (wall/ceiling)	Ignored silently — no log entry, no false alarm

> **Observed limitation:** matching quality depends on how large the face is in the frame. During phone testing at 640×480 the face was only ~55–90 px wide, which pushed the cosine score down to ~0.12 and below threshold. Holding the camera closer, or using a higher stream resolution (1280×720 or above), restores accurate recognition. This is a framing issue, not a model failure — the same face scores 0.48–0.67 when properly framed.

> **Design strength:** the system errs on the side of caution. A low-confidence face is logged as UNKNOWN and flagged for a human rather than being force-matched to the closest person. In the evaluation above, **zero false acceptances** occurred.

8. Security, Privacy & Ethical Considerations

Consent-first enrollment

Enrollment requires an explicit consent checkbox. No face is enrolled without it. Face embeddings are stored as binary blobs, not photographs.

Data locality

- All data lives locally: `database/vc_entry.db` plus snapshot images in `static/snapshots/`
- Nothing is uploaded to any cloud service or third party
- Full erasure = delete the database folder and snapshots folder
- Snapshots are capped (240 most recent) and pruned automatically

Human-in-the-loop by design

The software produces a *recommendation*, never an action. Every alert requires a human Acknowledge/Reject, and no physical access control is driven by the system.

Limitations and bias

- Accuracy depends on lighting, distance and face size in frame
- Trained primarily on adult faces — performance on very young or elderly subjects may differ
- The enrolled gallery determines who can ever be recognized; unknown faces are never guessed

What this prototype does NOT do

No door control, no lock control, no automated entry grant. The prototype stops at identification, logging and notification.

9. How to Run

```
python -m pip install -r requirements.txt
py app.py                              # webcam / sample data
```

```
py app.py --source visitor.mp4       # a video file
py app.py --source http://10.0.17.58:8080/video   # phone camera
```

Then open `http://127.0.0.1:5000`

Command	Purpose
`py app.py`	Start with the default source
`--source`	Webcam index, video file, image folder, or stream URL
`--port`	Change the dashboard port (default 5000)
`--no-toast`	Disable Windows desktop notifications
`py evaluate.py --self-test`	Re-run the accuracy evaluation
`py enrollment.py --image ... --name ...`	Enroll from the command line

Configuration

Variable	Default	Meaning
`VC_COOLDOWN`	15 (seconds)	Duplicate-entry suppression window
`VC_SOURCE`	auto	Override the default frame source
`VC_MAX_WIDTH`	960 px	Frames wider than this are downscaled before detection

10. Limitations & Future Work

Current limitation	Planned improvement
Single face tracked per frame (highest scoring)	Multi-face tracking with per-face independent decisions
Match quality sensitive to face size in frame	Higher-resolution capture + multi-frame scoring per visitor
Manual enrollment only	Bulk import from the college student database / ID cards
Flat cosine threshold	Per-person adaptive thresholds and liveness checks
Single machine, single camera	Multi-camera support across cabin entry points, with role-based logins
Local SQLite	Optional central server with an audit dashboard for security staff
Photo-only enrolment (no liveness)	Anti-spoofing — blink / head-turn liveness to reject photos and masks

11. Project Deliverables

#	Deliverable	Status
1	Face detection pipeline (YuNet)	Complete
2	Face recognition & matching (SFace + cosine)	Complete
3	Consent-gated enrollment	Complete
4	Visitor entry logging (SQLite)	Complete
5	Dashboard with live camera, logs, stats, alerts	Complete
6	Acknowledge / Reject with 24-hour auto-expiry	Complete
7	Windows toast notifications	Complete
8	Phone-as-webcam live streaming	Complete
9	Accuracy evaluation & metrics report	Complete
10	Source code on GitHub	Complete

Repository

```
https://github.com/Argyrys/Face-Recognition-System
```

> **Conclusion:** the prototype reliably detects and identifies visitors, produces a verifiable audit trail, and alerts staff in real time — while keeping the final access decision firmly in human hands.